

Software Architecture and Design Specification

Project: Traffic Management System (Simulation)

Software Architecture and Design Specification

Version: 1.0

S.No.	SRN	Name
1	PES2UG24CS672	Hariom N Kini
2	PES2UG24CS632	Deepthi S
3	PES2UG24CS653	Ramya
4	PES2UG24CS647	Peddinti Suhas

Revision History

Version	Date	Author	Change Summary
1.0	07-10-2026	Project Team	Initial Software Architecture and Design Specification for the Traffic Management System (Simulation).

Approvals

Role	Name	Signature/Date
Course Coordinator	Dr. Pradeep Kumar	
Project Team	Hariom N Kini, Deepthi S, Ramya, Peddinti Suhas	

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) describes the architecture and detailed design of the **Traffic Management System (Simulation)**. The system is a C/C++ console-based simulation that models vehicle movement through a four-way traffic intersection using traffic signals and FIFO vehicle queues.

The document describes the major system components, their responsibilities, interactions, architecture pattern, design interfaces, error handling, security considerations, and user interaction.

1.2 Scope

The system simulates a single four-way intersection consisting of **North, South, East, and West approaches**.

The system supports:

- Vehicle creation and automatic vehicle generation.
- Separate FIFO queues for each approach.
- Automatic and manual traffic signal control.
- Configurable green and yellow signal durations.
- Tick-based vehicle movement.
- Start, pause, resume, reset, and stop operations.
- Monitoring of queue lengths and signal states.
- Tracking of processed vehicles and waiting statistics.
- Calculation of average waiting time and throughput.
- Optional local text/CSV logging.

The system does not include real traffic hardware, sensors, cameras, network communication, multiple intersections, graphical interfaces, databases, machine-learning-based adaptive control, or real-world traffic calibration.

1.3 Audience

This document is intended for:

- Development team members.
- Test engineers and QA members.
- Project guide.
- Project evaluators.
- Students involved in maintaining or extending the system.

1.4 Definitions

Term	Definition
FIFO	First-In-First-Out queue behavior
Approach	One incoming direction of the intersection: North, South, East, or West
Traffic Phase	Current signal state controlling vehicle movement
Simulation Tick	One unit of simulated time during which the system updates its state
Throughput	Number of vehicles processed during the simulation
Vehicle Manager	Module responsible for creating and managing vehicles
Queue Manager	Module responsible for maintaining approach queues
Signal Manager	Module responsible for traffic signal states
Simulation Engine	Module controlling the overall simulation flow
Statistics Manager	Module responsible for collecting and calculating statistics

2. Document Overview

2.1 How to Use This Document

This document provides the architectural and design details required to understand and implement the Traffic Management System.

Section 3 describes the overall architecture, system components, technology choices, risks, requirement traceability, and security architecture.

Section 4 describes the detailed design, system interactions, module interfaces, error handling, and user interface.

2.2 Related Documents

The following documents are related to this SAD:

1. Software Requirements Specification (SRS) — Traffic Management System (Simulation)

2. Software Test Plan (STP) — Traffic Management System
3. Requirements Traceability Matrix (RTM) included in the SRS
4. Course/project guidelines provided for Software Engineering UE24CS341A

3. Architecture

3.1 Goals and Constraints

Architecture Goals

The architecture is designed to:

- Separate the responsibilities of different system modules.
- Maintain FIFO ordering of vehicles.
- Ensure safe traffic signal transitions.
- Prevent vehicle movement during red or yellow signals.
- Support automatic and manual signal control.
- Provide reliable and deterministic simulation behavior.
- Allow individual modules to be tested independently.
- Provide clear monitoring and statistics.

Constraints

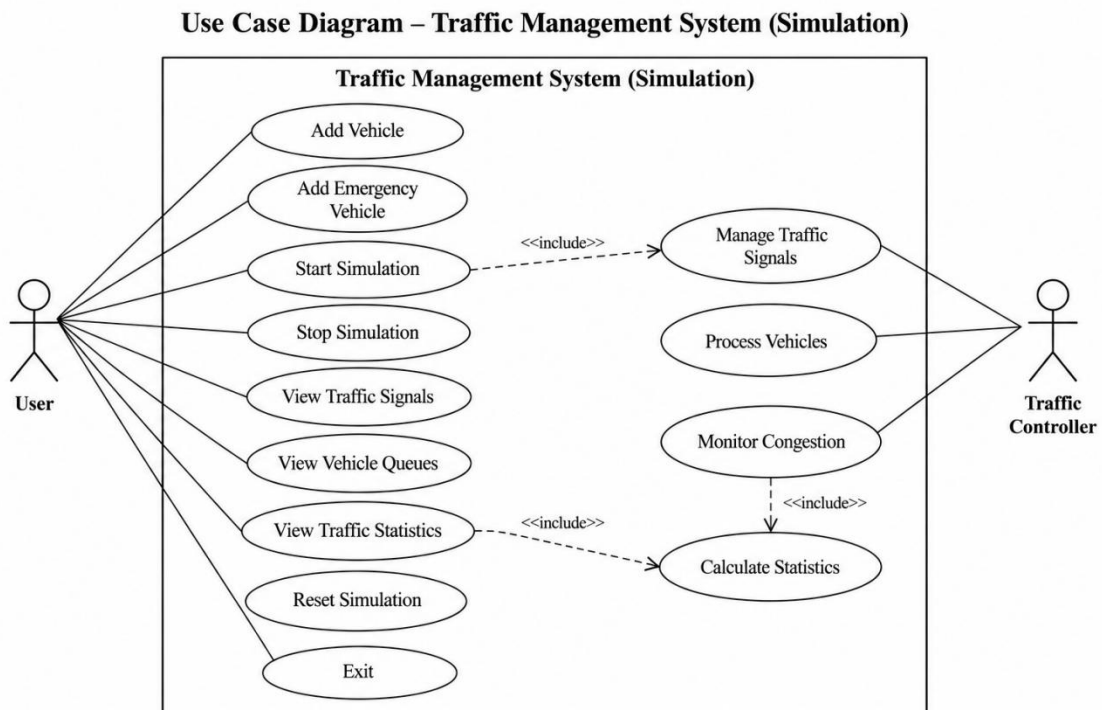
- The system supports only one four-way intersection.
- The system is console-based.
- The system does not use a database.
- The system does not require network connectivity.
- Traffic data is maintained locally in memory.
- Standard C/C++ libraries are used.
- The project is intended for implementation within a short student-project timeframe.
- The traffic model is simplified and does not represent all real-world traffic conditions.

3.2 Stakeholders and Concerns

Stakeholder	Main Concern
System Operator/User	Easy control and clear display of simulation state
Developers	Modularity and maintainability
Test/QA Team	Correct vehicle, queue, signal, and simulation behavior
Project Guide	Correctness and requirement coverage
Evaluators	Demonstrable and understandable simulation

3.3 Component / Class UML Diagram

UML DIAGRAM



3.4 Component Descriptions

Console Interface / Controller

Provides the user interface for controlling the simulation.

Responsibilities:

- Display menus and system status.
- Accept user input.
- Validate input.
- Start, pause, resume, reset, and stop simulation.
- Allow vehicle insertion.
- Allow signal mode selection.
- Display queues, signals, and statistics.

Vehicle Manager

Responsible for vehicle creation and management.

Responsibilities:

- Create vehicles.
- Generate unique vehicle IDs.
- Assign vehicles to approaches.
- Support automatic vehicle generation.
- Maintain basic vehicle information.

Queue Manager

Responsible for maintaining the four approach queues.

Responsibilities:

- Maintain separate queues for North, South, East, and West.
- Insert vehicles at the rear.
- Remove vehicles only from the front.
- Maintain FIFO behavior.
- Prevent invalid dequeue operations.
- Update queue counts.

Traffic Signal Manager

Responsible for traffic signal control.

Responsibilities:

- Maintain Red, Yellow, and Green states.
- Control automatic signal phases.
- Support manual signal control.
- Manage configurable green/yellow durations.
- Prevent conflicting signal phases.
- Prevent vehicle movement on red or yellow.

Simulation Engine

Controls the overall simulation.

Responsibilities:

- Run the tick-based simulation.
- Coordinate vehicles, queues, and signals.
- Process eligible vehicles.
- Update simulation time.
- Handle Start, Pause, Resume, Reset, and Stop operations.

Statistics Manager

Collects and calculates simulation statistics.

Responsibilities:

- Track waiting vehicles.
- Track processed vehicles.
- Calculate average waiting time.
- Calculate throughput.
- Generate the final simulation summary.

Logging / File Output Manager

Provides optional local file output.

Responsibilities:

- Save configuration information.
- Load configuration information.
- Optionally store final statistics in CSV format.
- Handle malformed files safely.

3.5 Chosen Architecture Pattern and Rationale

The system uses a **Modular / Layered Architecture**.

The system is divided into modules based on responsibility:

- User interaction is handled by the Console Interface.
- Overall control is handled by the Simulation Engine.
- Vehicle operations are handled by the Vehicle Manager.
- Queue operations are handled by the Queue Manager.
- Signal operations are handled by the Traffic Signal Manager.
- Statistics are handled by the Statistics Manager.
- Optional file operations are handled by the Logging/File Output Manager.

This architecture was selected because the project is a small local C/C++ simulation. It provides clear separation of responsibilities, makes individual modules easier to test, and simplifies maintenance and integration.

A distributed or microservice architecture is unnecessary because the system does not use network communication or multiple independent services.

3.6 Technology Stack and Data Stores

Category	Technology / Choice
Programming Language	C / C++
Recommended Standard	C++17

Compiler	GCC/G++, Clang, or MSVC
Interface	Console / Command Line
Data Structures	Queue, vector, array, deque or linked list
File Handling	Standard C/C++ file I/O
Operating System	Windows 10/11 or Linux
Database	Not used
Network	Not used
External APIs	Not used
Data Storage	In-memory data, optional text/CSV files

3.7 Risks and Mitigations

Risk	Mitigation
Incorrect FIFO queue operations	Perform queue unit testing
Invalid or conflicting signal phases	Validate signal transitions
Invalid user input	Implement input validation
Memory leaks	Use safe memory management and memory testing
Integration problems	Integrate modules incrementally
Limited implementation time	Prioritize core requirements first
Incorrect statistics	Test calculations using known simulation cases

3.8 Traceability to Requirements

Requirement	Design Component
TMS-F-001 to F-005	Vehicle Manager
TMS-F-006 to F-010	Queue Manager
TMS-F-011 to F-015	Traffic Signal Manager
TMS-F-016 to F-021	Simulation Engine
TMS-F-022 to F-026	Statistics Manager
TMS-F-027 to F-029	Logging / File Output Manager
TMS-NF-001 to NF-007	Applicable system components
TMS-SR-001 to SR-005	Console Interface, Queue Manager, File Manager

3.9 Security Architecture

Although the system is a local simulation and does not use accounts, networking, or sensitive user data, basic security and robustness measures are required.

The system provides:

- Input validation to prevent invalid commands and values.
- Safe handling of empty queues.
- Validation of signal transitions to prevent unsafe combinations.
- Validation of configuration-file values.
- Protection against invalid memory operations.
- Graceful handling of unexpected input and malformed files.
- Preservation of vehicle, queue, and signal state integrity.

4. Design

4.1 Design Overview

The system follows a modular design in which the **Console Interface** communicates with the **Simulation Engine**.

The Simulation Engine coordinates the Vehicle Manager, Queue Manager, Traffic Signal Manager, and Statistics Manager.

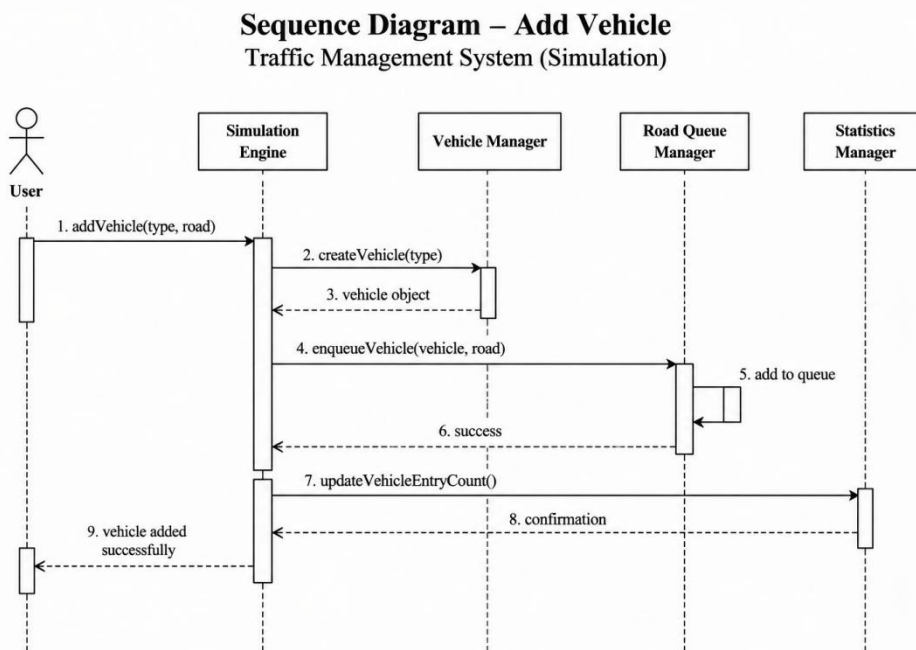
Vehicles are created and assigned to one of the four approach queues. The Queue Manager maintains FIFO ordering. The Traffic Signal Manager determines which approaches are allowed to move. During each simulation tick, the Simulation Engine checks the current signal state and processes eligible vehicles.

After a vehicle is processed, the Statistics Manager updates the relevant counters and waiting-time information. The current system state can then be displayed to the user.

4.2 UML Sequence Diagrams

4.2.1 Sequence Diagram — Add Vehicle

SEQUENCE DIAGRAM



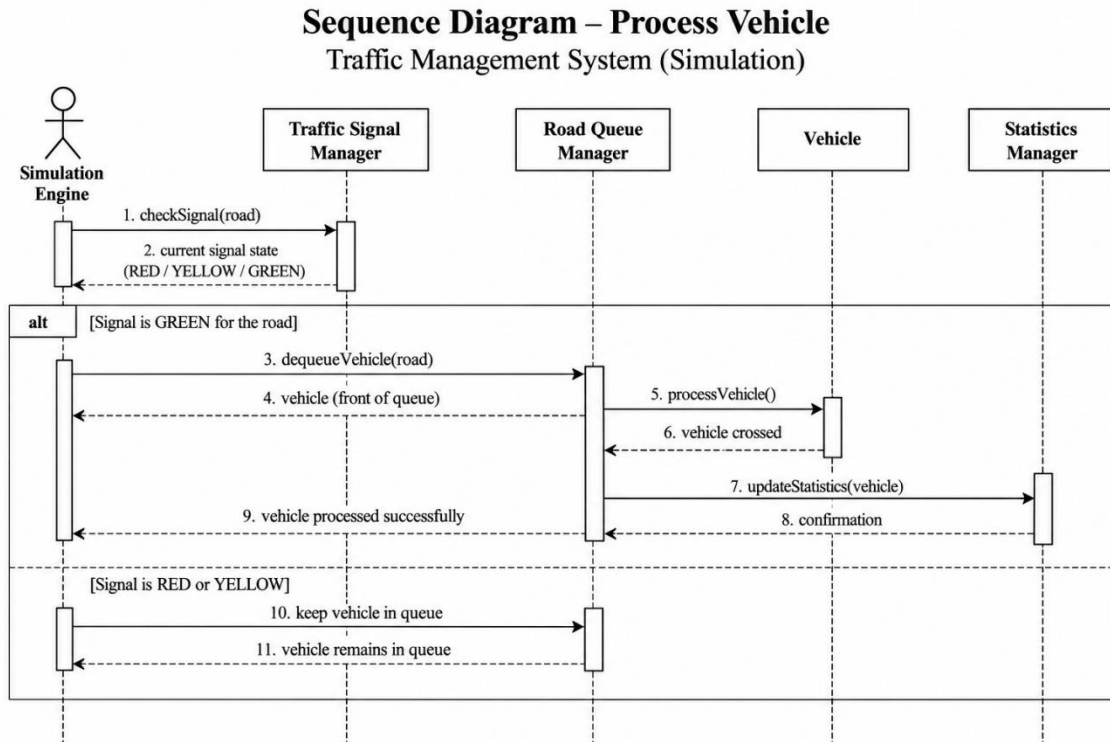
Description

When the user selects the Add Vehicle option, the Console Interface collects the required vehicle information and sends it to the Vehicle Manager. The Vehicle Manager creates a unique vehicle and assigns it to the selected approach. The Queue Manager then inserts the

vehicle at the rear of the corresponding FIFO queue. The updated queue status is displayed to the user.

4.2.2 Sequence Diagram — Process Vehicle

SECOND SEQUENCE DIAGRAM



Description

During each simulation tick, the Simulation Engine checks the current traffic signal through the Traffic Signal Manager. If an approach has a valid green signal, the Queue Manager checks the corresponding queue. The front vehicle is removed and processed only when movement is permitted. The Statistics Manager then updates the processed vehicle count, waiting-time information, and throughput.

4.3 API / Module Interface Design

Since this is a **C/C++ console application**, the system does not expose REST or web APIs. Instead, communication between modules is through functions and data structures.

Module	Example Functions	Purpose
Vehicle Manager	createVehicle()	Create a new vehicle

Vehicle Manager	<code>generateVehicle()</code>	Automatically generate a vehicle
Queue Manager	<code>enqueueVehicle()</code>	Add vehicle to queue
Queue Manager	<code>dequeueVehicle()</code>	Remove front vehicle
Queue Manager	<code>getQueueSize()</code>	Return queue length
Signal Manager	<code>getSignalState()</code>	Obtain current signal state
Signal Manager	<code>changeSignal()</code>	Change signal phase
Simulation Engine	<code>runSimulation()</code>	Run simulation
Simulation Engine	<code>resetSimulation()</code>	Reset simulation
Statistics Manager	<code>recordProcessedVehicle()</code>	Record processed vehicle
Statistics Manager	<code>calculateThroughput()</code>	Calculate throughput

4.4 Error Handling, Logging and Monitoring

The system handles errors without terminating unexpectedly.

Input Errors

Invalid menu selections, vehicle information, signal durations, and configuration values are rejected with a clear error message.

Queue Errors

The Queue Manager checks whether a queue is empty before attempting a dequeue operation.

Signal Errors

Invalid or conflicting signal combinations are prevented by the Traffic Signal Manager.

File Errors

Malformed or unavailable configuration files are handled gracefully without corrupting the current simulation state.

Monitoring

The console displays:

- Current signal states.
- Queue lengths.
- Waiting vehicle count.
- Processed vehicle count.
- Simulation tick.
- Final statistics.

4.5 UX Design

The system uses a **menu-driven console interface**.

The interface provides options such as:

1. Start Simulation
2. Pause Simulation
3. Resume Simulation
4. Reset Simulation
5. Stop Simulation
6. Add Vehicle
7. Enable/Disable Automatic Vehicle Generation
8. Configure Signal Durations
9. Select Automatic/Manual Signal Mode
10. View Current Queues and Signals
11. View Statistics

The interface provides clear status information after important operations and displays error messages when invalid input is entered.

4.6 Open Issues and Next Steps

Current Implementation Tasks

- Implement vehicle creation and generation.
- Implement four FIFO queues.
- Implement traffic signal management.
- Implement the simulation tick loop.
- Implement vehicle processing.
- Implement statistics calculation.
- Integrate all modules.
- Perform unit, integration, system, and acceptance testing.
- Verify memory safety and robustness.
- Implement optional file logging if time permits.

Possible Future Enhancements

The following features are outside the current scope but could be considered in future versions:

- Multiple intersections.
- Graphical user interface.
- Advanced lane-changing behavior.
- Pedestrian traffic.
- Public transport modeling.
- Adaptive traffic signal control.
- Real-world traffic data integration.
- Sensor or camera integration.

5. Appendices

5.1 Glossary

Term	Meaning
FIFO	First-In-First-Out

Queue	Data structure used to store waiting vehicles
Approach	Incoming direction of the intersection
Signal Phase	Current traffic signal state
Simulation Tick	One unit of simulation time
Throughput	Number of vehicles processed
N/S/E/W	North, South, East, and West approaches
STL	Standard Template Library in C++

5.2 References

1. Traffic Management System (Simulation) — Software Requirements Specification, Version 1.0.
2. Traffic Management System (Simulation) — Software Test Plan, Version 1.0.
3. Software Engineering UE24CS341A course/project guidelines.

5.3 Tools

- C/C++ compiler — GCC/G++, Clang, or MSVC
- Visual Studio Code / suitable C/C++ IDE
- Git/GitHub for version control
- diagrams.net / draw.io for UML diagrams
- PlantUML, if used for UML generation
- Valgrind or equivalent memory-analysis tool where available